

Отчет по лабораторной работе №8

Управление доступом, расширениями и локализацией

Дата: 2025-12-16

Семестр: 4 курс 1 полугодие – 7 семестр

Группа: ПИЖ-6-о-22-1

Дисциплина: Администрирование баз данных

Студент: Гойдь Алина Яновна

Цель работы

Освоить управление правами доступа пользователей, работу с расширениями PostgreSQL и настройку параметров локализации. Получить практические навыки настройки аутентификации, управления привилегиями, установки расширений и миграции данных между разными кодировками.

Теоретическая часть

Управление доступом: Система ролей и привилегий в PostgreSQL. Настройка аутентификации через файл `pg_hba.conf`.

Расширения: Способ упаковки и установки дополнительного функционала (типы данных, функции, операторы) в PostgreSQL.

Локализация: Настройки кодировки, правил сортировки (collation) и формата даты/времени.

Практическая часть

Часть 1. Управление доступом (Повторение)

Выполненные задачи:

1. Настроила привилегии: создал БД `access_db`, роли `writer` и `reader`, настроила `GRANT/REVOKE`, проверила доступ `w1` и `r1`.
2. Настроила аутентификацию через `pg_hba.conf` для ролей `alice` и `bob` с использованием методов `trust`, `md5` и `peer`.

Команды:

Задача 1:

```
CREATE DATABASE access_db;
\c access_db
CREATE ROLE writer NOLOGIN;
CREATE ROLE reader NOLOGIN;
```

```
REVOKE ALL ON SCHEMA public FROM PUBLIC;
GRANT USAGE, CREATE ON SCHEMA public TO writer;
GRANT USAGE ON SCHEMA public TO reader;
ALTER DEFAULT PRIVILEGES FOR ROLE writer IN SCHEMA public
    GRANT SELECT ON TABLES TO reader;
CREATE ROLE w1 LOGIN PASSWORD 'w1_pass';
CREATE ROLE r1 LOGIN PASSWORD 'r1_pass';
GRANT writer TO w1;
GRANT reader TO r1;
SET ROLE writer;
CREATE TABLE test_access (id INT PRIMARY KEY, data TEXT);
INSERT INTO test_access VALUES (1, 'test');
RESET ROLE;
```

```
sudo nano /etc/postgresql/16/main/pg_hba.conf
# local all postgres trust
# local all student trust
# local all alice peer map=users_map
# local all all md5
```

```
SELECT pg_reload_conf();
```

Фрагменты вывода:

```
CREATE DATABASE
You are now connected to database "access_db" as user "student".
CREATE ROLE
CREATE ROLE
REVOKE
GRANT
GRANT
ALTER DEFAULT PRIVILEGES
CREATE ROLE
CREATE ROLE
GRANT ROLE
GRANT ROLE
SET
CREATE TABLE
INSERT 0 1
RESET
```

Выполненные задачи:

1. Создала БД `backup_db` и таблицу с данными. Сделала логическую копию с помощью `pg_dump`. Удалила БД и восстановил её из копии. Убедилась в целостности данных.

2. Создала несколько БД с различными объектами. Сделала копию глобальных объектов через `pg_dumpall --globals-only`. Сделала дампы каждой БД с помощью `pg_dump` в параллельном режиме (`-j`).
3. Восстановила весь кластер из созданных резервных копий.
4. Создала ситуацию с ошибкой при загрузке дампа (несовпадение кодировок).

Команды:

Задача 1:

```
CREATE DATABASE backup_db;
\c backup_db
CREATE TABLE employees (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50),
    salary NUMERIC(10,2)
);
INSERT INTO employees (name, department, salary) VALUES
    ('Иванов Иван', 'IT', 75000.00),
    ('Петров Петр', 'Бухгалтерия', 65000.00),
    ('Сидорова Мария', 'HR', 60000.00);
```

```
pg_dump -d backup_db -Fc -f /tmp/backup_db.dump
```

```
DROP DATABASE backup_db;
CREATE DATABASE backup_db;
```

```
pg_restore -d backup_db /tmp/backup_db.dump
```

```
\c backup_db
SELECT * FROM employees;
```

Задача 2:

```
CREATE DATABASE prod_db1;
\c prod_db1
CREATE TABLE products (id SERIAL PRIMARY KEY, name TEXT, price NUMERIC);
INSERT INTO products VALUES (1, 'Товар А', 100), (2, 'Товар Б', 200);
```

```
CREATE DATABASE prod_db2;  
\c prod_db2  
CREATE TABLE orders (id SERIAL PRIMARY KEY, product_id INT, quantity INT);  
INSERT INTO orders VALUES (1, 1, 5), (2, 2, 3);
```

```
pg_dumpall --globals-only -f /tmp/globals.sql  
pg_dump -d prod_db1 -Fd -j 2 -f /tmp/prod_db1_dump  
pg_dump -d prod_db2 -Fd -j 2 -f /tmp/prod_db2_dump
```

Задача 3:

```
psql -f /tmp/globals.sql  
  
pg_restore -d prod_db1 -j 2 /tmp/prod_db1_dump  
pg_restore -d prod_db2 -j 2 /tmp/prod_db2_dump
```

Задача 4:

```
CREATE DATABASE test_encoding  
  WITH ENCODING 'LATIN1'  
  LC_COLLATE 'C'  
  LC_CTYPE 'C'  
  TEMPLATE template0;  
\c test_encoding  
CREATE TABLE test_data (id INT, text_data TEXT);  
INSERT INTO test_data VALUES (1, 'Test'), (2, 'Тест');
```

```
pg_dump -d test_encoding -f /tmp/test_encoding.sql
```

```
CREATE DATABASE restore_utf8 WITH ENCODING 'UTF8' TEMPLATE template0;
```

```
psql -d restore_utf8 -f /tmp/test_encoding.sql
```

Фрагменты вывода:

Задача 1:

```
CREATE DATABASE
You are now connected to database "backup_db" as user "postgres".
CREATE TABLE
INSERT 0 3
```

```
DROP DATABASE
CREATE DATABASE
```

id	name	department	salary
1	Иванов Иван	IT	75000.00
2	Петров Петр	Бухгалтерия	65000.00
3	Сидорова Мария	HR	60000.00

(3 rows)

Задача 2:

```
CREATE DATABASE
You are now connected to database "prod_db1" as user "postgres".
CREATE TABLE
INSERT 0 2

CREATE DATABASE
You are now connected to database "prod_db2" as user "postgres".
CREATE TABLE
INSERT 0 2
```

Задача 3:

You are now connected to database "prod_db1" as user "postgres".

id	name	price
1	Товар A	100
2	Товар B	200

(2 rows)

You are now connected to database "prod_db2" as user "postgres".

id	product_id	quantity
1	1	5
2	2	3

(2 rows)

Задача 4:

```
CREATE DATABASE
You are now connected to database "test_encoding" as user "postgres".
CREATE TABLE
INSERT 0 2
```

```
CREATE DATABASE
```

```
SET
SET
```

```
CREATE TABLE
psql:/tmp/test_encoding.sql:45: ERROR:  invalid byte sequence for encoding "UTF8":
0xd2 0xe5
CONTEXT:  COPY test_data, line 2
```

Модуль 3. Физическое резервное копирование и PITR**Выполненные задачи:**

1. Создала табличное пространство и БД с таблицей в нём. Сделала базовую резервную копию кластера с помощью `pg_basebackup` в формате tar со сжатием. Развернула второй кластер из этой копии, изменив расположение табличного пространства через `tablespace_map`.
2. Настроила непрерывную архивацию WAL на локальный каталог. Сделала базовую копию с помощью `pg_basebackup` (без WAL). Добавила данные в таблицу. Убедилась, что WAL ушел в архив. Восстановила второй сервер из резервной копии, применив архив WAL. Убедилась, что все данные восстановились. Остановила второй сервер и восстановил его снова до конкретного момента времени. Сравнила результаты.

Команды:**Задача 1:**

```
sudo mkdir -p /var/lib/postgresql/tablespaces/ts1
sudo chown postgres:postgres /var/lib/postgresql/tablespaces/ts1
```

```
CREATE TABLESPACE ts1 LOCATION '/var/lib/postgresql/tablespaces/ts1';
CREATE DATABASE basebackup_db TABLESPACE ts1;
```

```
\c basebackup_db
CREATE TABLE test_physical (id SERIAL PRIMARY KEY, data TEXT);
INSERT INTO test_physical (data)
SELECT 'Data ' || i FROM generate_series(1, 1000) i;
```

```
sudo mkdir -p /tmp/base_backup
sudo chown postgres:postgres /tmp/base_backup
pg_basebackup -D /tmp/base_backup -Ft -z -P
```

```
# Создание нового каталога для второго кластера
sudo mkdir -p /var/lib/postgresql/16/cluster2
sudo chown postgres:postgres /var/lib/postgresql/16/cluster2

# Распаковка базовой копии
cd /tmp/base_backup
sudo tar -xzf base.tar.gz -C /var/lib/postgresql/16/cluster2

# Создание нового расположения для табличного пространства
sudo mkdir -p /var/lib/postgresql/tablespaces/ts1_new
sudo chown postgres:postgres /var/lib/postgresql/tablespaces/ts1_new

# Редактирование tablespace_map
echo "16385 /var/lib/postgresql/tablespaces/ts1_new" | sudo tee
/var/lib/postgresql/16/cluster2/tablespace_map
```

Задача 2:

```
# Настройка WAL-архивации
sudo mkdir -p /var/lib/postgresql/wal_archive
sudo chown postgres:postgres /var/lib/postgresql/wal_archive
```

```
ALTER SYSTEM SET wal_level = replica;
ALTER SYSTEM SET archive_mode = on;
ALTER SYSTEM SET archive_command = 'test ! -f /var/lib/postgresql/wal_archive/%f
&& cp %p /var/lib/postgresql/wal_archive/%f';
```

```
sudo systemctl restart postgresql
```

```
\c basebackup_db
SELECT pg_switch_wal();
```

```
# Базовая резервная копия без WAL
pg_basebackup -D /tmp/pitr_backup -Ft -z -X none -P
```

```
INSERT INTO test_physical (data)
SELECT 'New data ' || i FROM generate_series(1001, 2000) i;
SELECT pg_switch_wal();
```

```
# Проверка архива WAL
ls -lh /var/lib/postgresql/wal_archive/
```

```
# Подготовка каталога для восстановления
sudo mkdir -p /var/lib/postgresql/16/pitr_cluster
cd /tmp/pitr_backup
sudo tar -xzf base.tar.gz -C /var/lib/postgresql/16/pitr_cluster

# Создание recovery.signal
sudo touch /var/lib/postgresql/16/pitr_cluster/recovery.signal

# Настройка recovery
sudo tee -a /var/lib/postgresql/16/pitr_cluster/postgresql.auto.conf << EOF
restore_command = 'cp /var/lib/postgresql/wal_archive/%f %p'
recovery_target_action = 'promote'
EOF
```

```
# Запуск восстановленного кластера на другом порту
sudo -u postgres /usr/lib/postgresql/16/bin/pg_ctl -D
/var/lib/postgresql/16/pitr_cluster -o "-p 5433" start
```

```
-- Подключение к восстановленному кластеру
psql -p 5433 -d basebackup_db
SELECT COUNT(*) FROM test_physical;
```

```
# Остановка для повторного восстановления
sudo -u postgres /usr/lib/postgresql/16/bin/pg_ctl -D
/var/lib/postgresql/16/pitr_cluster stop
```



```
# Очистка и повторная распаковка
sudo rm -rf /var/lib/postgresql/16/pitr_cluster/*
sudo tar -xzf /tmp/pitr_backup/base.tar.gz -C /var/lib/postgresql/16/pitr_cluster

# Настройка восстановления до определённого момента
sudo touch /var/lib/postgresql/16/pitr_cluster/recovery.signal
sudo tee /var/lib/postgresql/16/pitr_cluster/postgresql.auto.conf << EOF
restore_command = 'cp /var/lib/postgresql/wal_archive/%f %p'
recovery_target = 'immediate'
recovery_target_action = 'promote'
EOF
```

```
sudo -u postgres /usr/lib/postgresql/16/bin/pg_ctl -D
/var/lib/postgresql/16/pitr_cluster -o "-p 5433" start
```

```
psql -p 5433 -d basebackup_db
SELECT COUNT(*) FROM test_physical;
```

Фрагменты вывода:

Задача 1:

```
CREATE TABLESPACE
CREATE DATABASE
You are now connected to database "basebackup_db" as user "postgres".
CREATE TABLE
INSERT 0 1000
```

```
441263/441263 kB (100%), 2/2 tablespace
```

Задача 2:

```
ALTER SYSTEM
ALTER SYSTEM
ALTER SYSTEM
```

```
pg_switch_wal
-----
0/3000098
(1 row)
```

```
INSERT 0 1000
pg_switch_wal
-----
0/4000110
(1 row)
```

```
total 48M
-rw----- 1 postgres postgres 16M Dec 12 12:30 0000000100000000000000003
-rw----- 1 postgres postgres 16M Dec 12 12:31 0000000100000000000000004
```

```
waiting for server to start.... done
server started
```

```
You are connected to database "basebackup_db" as user "postgres" on host
"localhost" (address "127.0.0.1") at port "5433".
```

```
count
-----
2000
(1 row)
```

```
count
-----
1000
(1 row)
```

Модуль 4. Обновление сервера (Логический метод)

Выполненные задачи:

1. Подготовила данные в PostgreSQL 16 (смоделировал обновление с версии 15): создала пользователя и БД с таблицей и данными.
2. Создала логическую резервную копию БД средствами `pg_dump`.
3. В кластере PostgreSQL 16 создала табличное пространство. Восстановила дамп, указав целевое табличное пространство для данных.

4. Проверила работоспособность сервера, доступность пользователя и целостность данных после обновления.

Команды:

Задача 1:

```
-- Моделирование PostgreSQL 15 (используем PG16)
CREATE USER olduser WITH PASSWORD 'oldpass';
CREATE DATABASE old_version_db OWNER olduser;
\c old_version_db
CREATE TABLE legacy_data (
    id SERIAL PRIMARY KEY,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    info TEXT
);
INSERT INTO legacy_data (info) VALUES
    ('Запись из старой версии 1'),
    ('Запись из старой версии 2'),
    ('Запись из старой версии 3');
GRANT ALL ON TABLE legacy_data TO olduser;
GRANT USAGE, SELECT ON SEQUENCE legacy_data_id_seq TO olduser;
```

Задача 2:

```
pg_dump -d old_version_db -Fc -f /tmp/migration_dump.dump
```

Задача 3:

```
sudo mkdir -p /var/lib/postgresql/tablespaces/ts_migration
sudo chown postgres:postgres /var/lib/postgresql/tablespaces/ts_migration
```

```
CREATE TABLESPACE ts_migration LOCATION
'/var/lib/postgresql/tablespaces/ts_migration';
CREATE DATABASE new_version_db OWNER olduser TABLESPACE ts_migration;
```

```
pg_restore -d new_version_db /tmp/migration_dump.dump
```

Задача 4:

```
\c new_version_db
SELECT * FROM legacy_data;
```

```
\dt+
```

```
psql -U olduser -d new_version_db
```

```
SELECT * FROM legacy_data;
INSERT INTO legacy_data (info) VALUES ('Новая запись после миграции');
SELECT * FROM legacy_data;
```

Фрагменты вывода:

Задача 1:

```
CREATE ROLE
CREATE DATABASE
You are now connected to database "old_version_db" as user "postgres".
CREATE TABLE
INSERT 0 3
GRANT
GRANT
```

Задача 3:

```
CREATE TABLESPACE
CREATE DATABASE
```

Задача 4:

```
You are now connected to database "new_version_db" as user "postgres".

 id |          created_at          | info
----+-----+-----+-----
  1 | 2025-12-13 01:43:39.971842 | Запись из старой версии 1
  2 | 2025-12-13 01:43:39.971842 | Запись из старой версии 2
  3 | 2025-12-13 01:43:39.971842 | Запись из старой версии 3
(3 rows)
```

```
                                List of relations
 Schema | Name      | Type  | Owner  | Persistence | Access method | Size  |
-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
```

```
public | legacy_data | table | postgres | permanent | heap | 16 kB |
(1 row)
```

Password for user olduser:

psql (16.4 (Ubuntu 16.4-1.pgdg24.04+1))

You are now connected to database "new_version_db" as user "olduser".

```
id |          created_at          | info
---+-----+-----
 1 | 2025-12-13 01:43:39.971842 | Запись из старой версии 1
 2 | 2025-12-13 01:43:39.971842 | Запись из старой версии 2
 3 | 2025-12-13 01:43:39.971842 | Запись из старой версии 3
(3 rows)
```

INSERT 0 1

```
id |          created_at          | info
---+-----+-----
 1 | 2025-12-13 01:43:39.971842 | Запись из старой версии 1
 2 | 2025-12-13 01:43:39.971842 | Запись из старой версии 2
 3 | 2025-12-13 01:43:39.971842 | Запись из старой версии 3
 4 | 2025-12-13 01:46:20.742078 | Новая запись после миграции
(4 rows)
```

Результаты выполнения

1. Управление доступом

Успешно повторены задания по настройке привилегий и аутентификации. Созданы роли **writer** и **reader** с соответствующими правами доступа. Настроены методы аутентификации в **pg_hba.conf** для различных пользователей.

2. Логическое резервное копирование

Выполнено логическое резервное копирование с использованием утилит **pg_dump** и **pg_dumpall**. Основные наблюдения:

Простой дамп и восстановление:

- Утилита **pg_dump** с форматом custom (**-Fc**) создает компактные сжатые дампы
- Восстановление через **pg_restore** позволяет выборочно восстанавливать объекты
- Все данные и структура таблиц восстановились полностью

Параллельный дамп:

- Опция **-j** позволяет использовать несколько потоков для ускорения дампа
- Формат directory (**-Fd**) необходим для параллельного дампа
- **pg_dumpall --globals-only** сохраняет глобальные объекты (роли, табличные пространства)

Проблемы при загрузке:

- Несовпадение кодировок может вызвать ошибки при восстановлении
- Важно проверять совместимость параметров исходной и целевой БД

Логическое копирование гибкое и позволяет восстанавливать данные между разными версиями PostgreSQL, но может быть медленным для больших БД.

3. Физическое резервное копирование и PITR

Освоено физическое резервное копирование и восстановление на определённый момент времени.

Базовая резервная копия:

- `pg_basebackup` создает полную копию кластера на уровне файловой системы
- Формат `tar (-Ft)` со сжатием (`-z`) экономит дисковое пространство
- `tablespace_map` позволяет изменить расположение табличных пространств при восстановлении
- Физическая копия быстрее логической для больших объёмов данных

PITR (Point-in-Time Recovery):

- Настройка непрерывной архивации WAL через `archive_command` обеспечивает сохранение всех изменений
- Параметры `wal_level = replica` и `archive_mode = on` активируют архивацию
- Файл `recovery.signal` указывает PostgreSQL выполнить восстановление
- `restore_command` определяет, как получать архивные WAL-файлы
- `recovery_target` позволяет восстановить БД на конкретный момент времени

Сравнение режимов восстановления:

- Восстановление без `recovery_target` применяет все доступные WAL-файлы (2000 записей)
- Восстановление с `recovery_target = 'immediate'` останавливается сразу после достижения консистентности (1000 записей)
- PITR критически важен для восстановления после логических ошибок или удаления данных

Физическое копирование быстрее и обеспечивает точное восстановление состояния кластера, но требует больше настройки и дискового пространства для архивов WAL.

4. Обновление сервера

Выполнена миграция данных между версиями PostgreSQL с использованием логического метода.

Процесс обновления:

1. Создание полного дампа исходной БД с помощью `pg_dump`
2. Создание табличного пространства в новой версии
3. Восстановление дампа в целевую БД с указанием табличного пространства
4. Проверка целостности данных и доступности пользователей

Ключевые моменты:

- Логический дамп совместим между мажорными версиями PostgreSQL

- Права доступа пользователей сохраняются в дампе
- После миграции пользователь может работать с данными без дополнительной настройки
- Табличное пространство можно указать при создании БД или изменить позже

Выводы

1. **Логическое и физическое копирование:** Освоила оба подхода к резервному копированию. Логическое копирование (`pg_dump`, `pg_dumpall`) гибкое и совместимо между версиями, но медленнее. Физическое копирование (`pg_basebackup`) быстрее и создаёт точную копию кластера, но требует одинаковой версии PostgreSQL и архитектуры.
2. **PITR:** Поняла важность непрерывной архивации WAL для восстановления на произвольный момент времени. Настроил параметры `wal_level`, `archive_mode` и `archive_command`. Научилась использовать `recovery_target` для точного управления процессом восстановления.
3. **Параллелизм:** Использование опции `-j` в `pg_dump` и `pg_restore` значительно ускоряет операции для больших БД. Формат `directory` необходим для параллельной работы.
4. **Обновление версий:** Логический метод обновления через дампы/восстановление — надёжный способ миграции между мажорными версиями PostgreSQL. Важно сохранять глобальные объекты отдельно через `pg_dumpall --globals-only`.
5. **Табличные пространства:** Научилась работать с табличными пространствами при создании копий и восстановлении. Использование `tablespace_map` позволяет изменить расположение данных при физическом восстановлении.
6. **Практические навыки:** Получила опыт работы с утилитами `pg_dump`, `pg_dumpall`, `pg_basebackup`, `pg_restore`. Поняла разницу между форматами дампов (`plain`, `custom`, `directory`, `tar`) и их применение в различных сценариях.